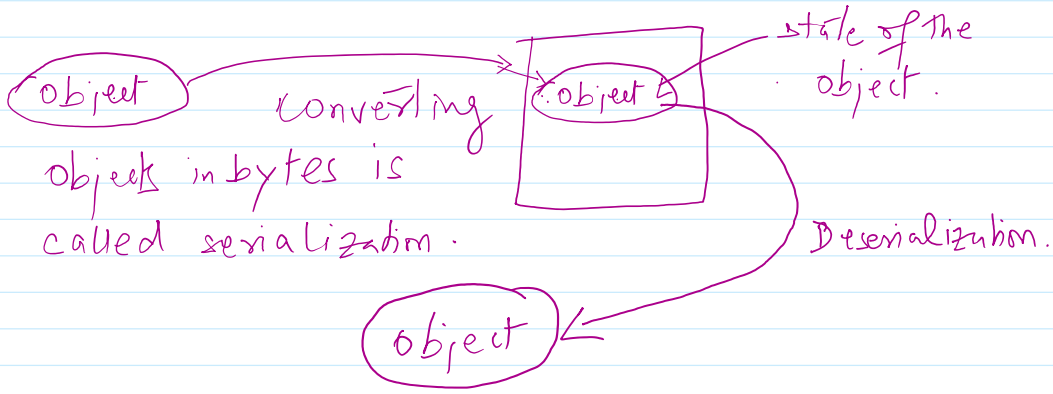
Serialization And Deserialization

17 August 2024 20:01

Serialization and deserialization are key concepts in Java. Often used in the context of saving objects to file and sending objects over network. or persisting an object state in a format that can be restored later.

1. **Serialization:** Serialization is the process of converting an object into byte stream, which can then be easily saved to a file sent over a network or stored in a database. The byte stream represents the object state, including its data field, and can be reconstructed later to recreate the object.



Key points:

- **Purpose:** Serialization is used to persist the state of an object so that it can be recreated later or sent to another environment.
- **Serializable Interface:** In Java, foreign object to be serialized Its class must implement the **'serializable'** interface. This is a marker interface(It does not have any methods.) that tells the Java virtual machine (JVM) That the object is eligible for serialization.
- **'ObjectOutputStream' Class**: This class is used to write the object to an output stream. Example, a file or a network socket. The **'writeObject()'** Method of this class is used to serialize an object.
- **Transient fields:** If you do not want a particular field to be serialized, you can mark it with the **'transient'** keyword. Transient fields are not included in the serialized state.

Deserialization

Deserialization is the reverse process of serialization. It involves converting a byte stream back into a copy of the original object, restoring its state.

- **Purpose:** Deserialization is used to recreate an object from its serialized form. Restoring the state of the object.
- **'ObjectInputStream' class:** This class is used to read the object from an input stream (Example, a file or a network socket) The **'readObject()'** method. of this class is used to deserialize an object.
- **Serial version UID: The 'serialVersionUID'** Is a unique identifier for each class that is serialized. It is. used during deserialization to verify that the sender and the receiver of a serialized object have loaded classes for that object that are compatible with respect to serialization. If the **'serialVersionUID'** of the class does not match that of the serialized object, it will throw an **'InvalidClassException'**.

```
package FileHandling1.src;
import java.io.FileInputStream;
import java.io.ObjectInputStream;
public class Deserialization {
    public static void main(String[] args) {
        //Deserialization
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("./DataFiles/person.ser"))) {
```

```
            Person p = (Person)ois.readObject();
            System.out.println("Object has been deserialized");
            System.out.println("Age: " +p.getAge());
            System.out.println("Name: "+p.getName());
            System.out.println("Your password: "+ p.getPassword());
        } catch (Exception e) {
            // TODO: handle exception
            e.printStackTrace();
        }
    }
}
```

Output:

```
Object has been deserialized
Age: 40
Name: Harry
Your password: null
```

password is transient. so it is null.

Customizing Serialization

Java allows customization of the serialization process by defining two special methods in the class:

- **'private void writeObject(ObjectOutputStream oos) throws IOException':** This method can be used to control how the object is serialized. If present, it is invoked by the serialization process instead of the default mechanism.
- **'private void readObject(ObjectInputStream ois) throws IOException':** This method can be used to control how the object is deserialized if present. It is invoked by the deserialization process instead of the default mechanism.

These methods provide flexibility, allowing you to perform additional processing such as encrypting data before serialization or performing validation during deserialization.

Serial Version UID

The 'serialVersionUID' Is a special field that you can declare in your class to control the versioning of serialized data. It is used to ensure that the same class version is used during both serialization and deserialization. If the **'serialVersionUID'** is not declared, Java will generate one based on the class details, which can change if the class structure changes leading to potential compatibility issues.

private static final long serialVersionUID = 1L;

Declaring a **'serialVersionUID'** explicitly ensures that the class remains compatible across different versions of the software.

Exceptions in Serialization/Deserialization

- **NotSerializableException:** Throne if an object that does not implement **'Serializable'** is attempted to be serialized.
- **InvalidClassException:** Thrown if there is a mismatch between the **'serialVersionUID'** of the class and the serialized object during deserialization.
- **OptionalDataException:** Throne when primitive data types are found in the stream instead of objects during deserialization.

Use Cases

- **Persistence:** Saving objects to file or databases for later retrieval.
- **Communication:** Sending objects over a network between different Java application (Example, remote method invocation (RMI)).
- **Caching:** Storing object in a serialized form for fast retrieval.

Custom Serialization

Custom serialization allows you to control the serialization and deserialization process. This is useful when

you need to serialize object in a non-standard way, or include/exclude certain fields dynamically.

Implementing custom serialization:
You can customize the serialization process by defining right object and read object methods in your class. These methods will be invoked by the JVM during serialization and deserialization, instead of the default behavior.

```
package FileHandling1.src;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
public class Account implements Serializable {
    private static final long serialVersionUID = 1L;
    private String username;
    private transient String password;
    public Account(String username, String password) {
        this.username = username;
        this.password = password;
    }
    private void writeObject(ObjectOutputStream oos) throws IOException{
        oos.defaultWriteObject(); //Write non-transient fields
        String encryptedPassword = "123"+password; //simple encryption
        oos.writeObject(encryptedPassword);
    }
    private void readObject(ObjectInputStream ois) throws IOException,
ClassNotFoundException{
        ois.defaultReadObject(); //read non-transient fields
        String encryptedPassword = (String)ois.readObject();
        this.password = encryptedPassword.substring(3); //simple decryption
    }
    @Override
    public String toString() {
        return "username=" + username +
        "\npassword="+password;
    }
}
```

```
package FileHandling1.src;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
public class CustomSerialization {
    public static void main(String[] args) {
        //Serialization
        try (ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream("./DataFiles/account.ser") )) {
            Account ob = new Account("user123", "mypassword");
            oos.writeObject(ob);

        } catch (IOException e) {
            // TODO: handle exception
            e.printStackTrace();
        }
```

```java
        //Deserialization
        try (ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream("./DataFiles/account.ser")) ) {
                Account ob = (Account)ois.readObject();
            System.out.println("Deserialized Account");
            System.out.println(ob);
        } catch (IOException | ClassNotFoundException e) {
            // TODO: handle exception
            e.printStackTrace();
        }
    }
}
```

Output:

```
Deserialized Account
username=user123
password=mypassword
```

- The 'writeobject()' method encrypts the password before serialization.
- The 'readObject()' method decrypts the password during deserialization.
- The 'defaultWriteObject()' and 'defaultReadObject()' methods handle the default serialization for non-transient fields.